Python语言与系统设计

（面向对象程序设计）

Python Language & System Design

第6章 面向对象程序设计

(Object oriented Programming)



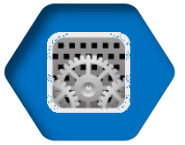
合肥工业大学 计算机与信息学院

第6章 面向对象程序设计


6.1 概述


6.2 类的定义与使用


6.3 方法


6.4 属性


6.5 继承机制


6.6 多态原理与实现


6.7 本章小结



6.1 概述

面向对象程序设计

- Python完全采用了面向对象程序设计的思想，是真正面向对象的高级动态编程语言，完全支持面向对象的基本功能，如封装、继承、多态以及对基类方法的覆盖或重写。

- Python中对象的概念很广泛，Python中的一切内容都可以称为对象，除了数字、字符串、列表、元组、字典、集合、range对象、zip对象等等，函数也是对象，类也是对象。

- 创建类时用变量形式表示的对象属性称为数据成员，用函数形式表示的对象行为称为成员方法，成员属性和成员方法统称为类的成员。

6.2 类的定义与使用

类定义语法

- Python使用class关键字来定义类
  - ✓ class关键字之后是一个空格，然后是类的名字，再然后是一个冒号，最后换行并定义类的内部实现。
- 类名的首字母一般要大写
  - ✓ 当然也可以按照自己的习惯定义类名，但一般推荐参考惯例来命名，并在整个系统的设计和实现中保持风格一致。

```
class Car:
    def infor(self):
        print(" This is a car ")
```

6.2 类的定义与使用

- 类定义后，可以用来实例化对象，并通过“对象名.成员”的方式来访问其中的数据成员或成员方法。

```
>>> car = Car()
>>> car.infor()
 This is a car
```

- Python中可以使用内置方法isinstance()来测试一个对象是否为某个类的实例。

```
>>> isinstance(car, Car)
True
>>> isinstance(car, str)
False
```

- **Python提供了一个关键字“pass”，表示空语句**
  - ✓ 可用在类和函数的定义中或者选择结构中
  - ✓ 当暂时没确定如何实现功能，或为以后软件升级预留空间等用其“占位”。

```
>>> class A:
    pass
```

```
>>> def demo():
    pass
```

```
>>> if 5>3:
    pass
```

6.2 类的定义与使用

self参数

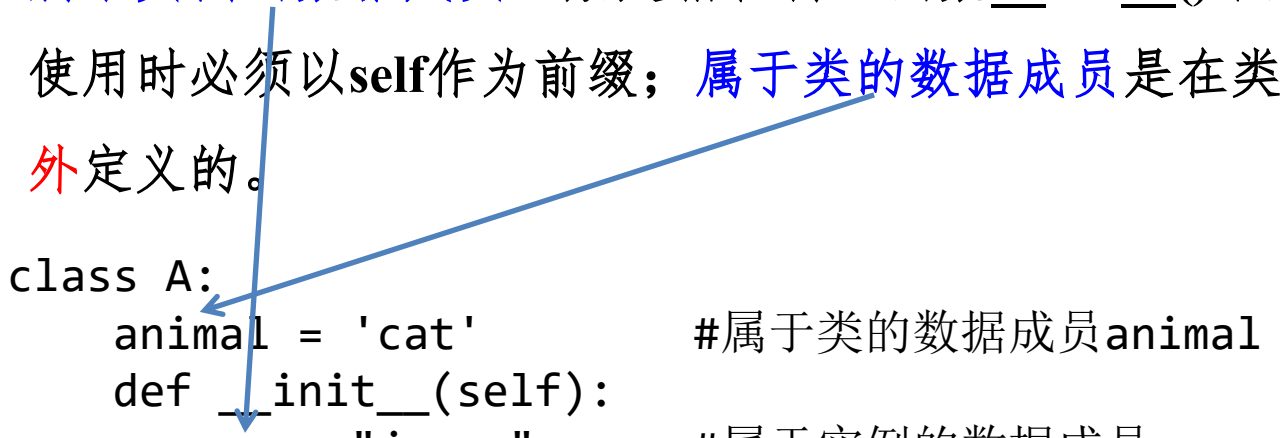
- 类的所有实例方法都必须至少有一个名为self的参数，且必须是方法的第一个形参（如果有多个形参的话），self参数代表将来要创建的对象本身。
- 在类的实例方法中访问实例属性时需以self为前缀。
- 在外部通过对象调用对象方法时并不需要传递这个参数，如果在外部通过类调用对象方法则需要显式为self参数传值。
- 定义为“self”只是一个习惯，可以换其它名字。

```
>>> class A:
    def __init__(self, v): #可以把self换成其它名字，比如haha
        self.value = v
    def show(self):
        print(self.value)
>>> a = A(3)
>>> a.show()  #通过对象调用不需要传递self参数
3
>>> A.show(a) #通过类调用对象方法则需要显式为self参数传值
```

6.2 类的定义与使用

类成员与实例成员

- 属于实例的数据成员一般是指在构造函数__init__()中定义的，定义和使用时必须以self作为前缀；属于类的数据成员是在类中所有方法之外定义的。



```
class A:
    animal = 'cat'              #属于类的数据成员animal
    def __init__(self):
        name="jerry"            #属于实例的数据成员name
A.age=2     #动态添加成员
```

- Python中可以动态地为自定义类和对象增加或删除成员
  - 此点不同于其他面向对象程序设计语言，也是Python动态类型特点的一种重要体现。

6.2 类的定义与使用

类属性，又称为静态字段。

实例属性，又称为普通字段。

■实例属性属于实例(对象)，只能通过对象名访问；

而类属性属于类，可以通过类名或对象名都可以访问。

```
class A:
  # 静态字段, 类属性
   animal = 'cat'
   def __init__(self, name):
      # 普通字段，实例属性
      self.name = name
obj = A('jerry')
print(obj.name)   #通过对象名访问

print(A.animal)   #通过类名或对象名都可以访问
print(obj.animal)
```

静态字段在内存中只保存一份（保存在类中）

普通字段在每个对象中都要保存一份（保存在对象中）

6.2 类的定义与使用

- 类成员与实例成员---示例

```
class Car:
    price = 100000                          #定义类属性
    def __init__(self, c):
        self.color = c                      #定义实例属性
car1 = Car("Red")                           #实例化对象
car2 = Car("Blue")
print(car1.color, Car.price)                #查看实例属性和类属性的值
Car.price = 110000                          #修改类属性
Car.name = 'QQ'                             #动态增加类属性
car1.color = "Yellow"                       #修改实例属性
print(car2.color, Car.price, Car.name)
print(car1.color, Car.price, Car.name)#price，name是不变的，color可变

import types          #types模块中MethodType动态的给对象添加实例方法：
def setSpeed(self, s):
    self.speed = s
car1.setSpeed = types.MethodType(setSpeed, car1) #动态增加成员方法
car1.setSpeed(50)                                #调用成员方法
print(car1.speed)     #注意car2对象并没有添加setSpeed
```

6.2 类的定义与使用

类成员与实例成员

- **Python类型的动态性使得我们可以动态为自定义类及其对象增加新的属性和行为，俗称混入（mixin）机制，这在大型项目开发中会非常方便和实用。**
  - ✓ 例如：系统中的所有用户分类非常复杂，不同用户组具有不同的行为和权限，且可能会经常改变。这时可以独立地定义一些行为，然后根据需要来为不同的用户设置相应的行为能力。

```
import types
class Person(object):
    def __init__(self, name):
        self.name =name

def sing(self):  #注意：这里定义的是外部函数
        print(self.name+' can sing.')
def walk(self):
        print(self.name+' can walk.')
```

```
>>> zhang = Person('zhang')
>>> zhang.sing()                                    #用户不具有该行为
AttributeError: 'Person' object has no attribute 'sing'
>>> zhang.sing = types.MethodType(sing, zhang)     #动态增加一个新行为
>>> zhang.sing()
zhang can sing.

>>> zhang.walk = types.MethodType(walk, zhang)
>>> zhang.walk()
zhang can walk.
>>> del zhang.walk                                  #删除用户行为
>>> zhang.walk()
AttributeError: 'Person' object has no attribute 'walk'
```

6.2 类的定义与使用

- 私有成员与公有成员
  - Python并没有对私有成员提供严格的访问保护机制。
    - ✓ 在定义类的成员时，如果成员名以两个下划线“__”或更多下划线开头而不以两个或更多下划线结束则表示是私有成员。
    - ✓ 私有成员在类的外部不能直接访问，需要通过调用对象的公开成员方法来访问，也可以通过Python支持的特殊方式来访问。
  - 公有成员既可以在类的内部进行访问，也可以在外部程序中使用。

```
>>> class A:
    def __init__(self, value1 = 0, value2 = 0):
        self._value1 = value1
        self.__value2 = value2
    def show(self):
        print(self._value1)
        print(self.__value2)

>>> a = A()
>>> a._value1
0
>>> a._A__value2    #在外部访问对象的私有数据成员
0
```

6.2 类的定义与使用

- Python中以下划线开头的变量名和方法名有特殊含义，尤其是在类的定义中。
  - _xxx：受保护成员，不能用'from module import *'导入；
  - __xxx__：系统定义的特殊成员；
  - __xxx：私有成员，只有类对象自己能访问，子类对象不能直接访问到这个成员，但在对象外部可以通过“对象名._类名__xxx”这样的特殊方式来访问。

✓ 注意：Python中不存在严格意义上的私有成员。

✓ 演示私有成员定义和访问的方法：

```
>>> class Fruit:
    def __init__(self):
        self.__color = 'Red'
        self.price = 1

>>> apple = Fruit()
>>> apple.price                                  #显示对象公有数据成员的值
1
>>> apple.price = 2                              #修改对象公有数据成员的值
>>> apple.price
2
>>> print(apple.price, apple._Fruit__color)      #显示对象私有数据成员的值
2 Red
>>> apple._Fruit__color = "Blue"                 #修改对象私有数据成员的值
>>> print(apple.price, apple._Fruit__color)
2 Blue
>>> print(apple.__color)          #不能直接访问对象的私有数据成员，出错
AttributeError: Fruit instance has no attribute '__color'
```

6.2 类的定义与使用

- 在IDLE交互模式下，一个下划线“_”表示解释器中最后一次显示的内容或最后一次语句正确执行的输出结果。

```
>>> 8 + 2
10
>>> _ * 3
30
>>> _ / 5
6.0
>>> _
6.0
```

- 在程序中，可以使用一个下划线来表示不关心该变量的值。

```
>>> for _ in range(5):
    print(3, end=' ')

3 3 3 3 3
```

6.3 方法

在类中定义的方法可以粗略分为：

- ✓ 普通方法：由对象调用；至少一个self参数；执行普通方法时，自动将调用该方法的对象赋值给self;
- ✓ 类方法： 至少一个cls参数；执行类方法时，自动将调用该方法的类复制给cls； 用修饰器@classmethod 声明类方法
- ✓ 静态方法： 无默认参数；用修饰器@staticmethod声明静态方法



6.3 方法

✓ 一般将cls作为类方法的第一个参数名称，但也可以使用其他的名字作为参数。

```
class Root:
    __total = 0
    def __init__(self, v):      #构造方法
        self.__value = v
        Root.__total += 1
    def show(self):             #普通实例方法
        print('self.__value:', self.__value)
        print('Root.__total:', Root.__total)
    @classmethod                #修饰器，声明类方法
    def classShowTotal(cls):
        print(cls.__total)
    @staticmethod               #修饰器，声明静态方法
    def staticShowTotal():
        print(Root.__total)
```

```
>>> r = Root(3)
>>> r.classShowTotal()                    #通过对象来调用类方法
1
>>> r.staticShowTotal()                   #通过对象来调用静态方法
1
>>> r.show()
self.__value: 3
Root.__total: 1
>>> rr = Root(5)
>>> Root.classShowTotal()                 #通过类名调用类方法
2
>>> Root.staticShowTotal()                #通过类名调用静态方法
2
>>> Root.show()     #试图通过类名直接调用实例方法，失败
TypeError: unbound method show() must be called with Root instance as
first argument (got nothing instead)
>>> Root.show(rr)  #通过类名调用实例方法时为self参数显式传递对象名
self.__value: 5
Root.__total: 2
```

常用特殊方法

- Python类有大量的特殊方法，其中比较常见的是构造函数和析构函数，除此之外，Python还支持大量的特殊方法，运算符重载就是通过重写特殊方法实现的。

✓ Python中类的构造函数是__init__()，一般用来为数据成员设置初值或进行其他必要的初始化工作，在创建对象时被自动调用和执行。如果用户没有设计构造函数，Python将提供一个默认的构造函数用来进行必要的初始化工作。

✓ Python中类的析构函数是__del__()，一般用来释放对象占用的资源，在Python删除对象和收回对象空间时被自动调用和执行。如果用户没有编写析构函数，Python将提供一个默认的析构函数进行必要的清理工作。

常用特殊方法

方法	功能说明
__new__()	类的静态方法，用于确定是否要创建对象
__init__()	构造方法，创建对象时自动调用
__del__()	析构方法，释放对象时自动调用
__add__()	+
__sub__()	-
__mul__()	*
__truediv__()	/
__floordiv__()	//
__mod__()	%
__pow__()	**
__eq__()、__ne__()、__lt__()、__le__()、__gt__()、__ge__()	==、!=、<、<=、>、>=
__lshift__()、__rshift__()	<<、>>
__and__()、__or__()、__invert__()、__xor__()	&、|、~、^



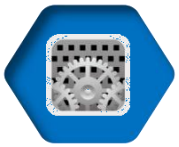
6.3 方法

特殊方法举例:__str__

如果一个类中定义了__str__方法，那么在打印 对象 时，默认输出该方法的返回值。

```
class Foo:
    def __str__(self):
        return 'this is a book'


obj = Foo()
print (obj)
# 输出: this is a book
```



6.4 属性

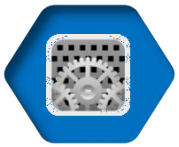
- **Python**中的属性其实是普通方法的变种。

定义时，在普通方法的基础上添加 **@property** 装饰器；属性仅有一个**self**参数

调用时，无需括号。

属性存在意义是：访问属性时可以制造出和访问字段完全相同的假象

```
class A:
    def  func(self):
        pass
    @property
    def  show(self):
        return "is ok"
a1 = A()
result=a1.show    #属性，不加括号（）
print(result)
a1.func()    #方法
```



6.4 属性

- 属性的定义有两种方式：

装饰器方式：在类的普通方法上应用@property装饰器

静态字段方式，创建值为property对象的静态字段

```
class Foo:
    def get_bar(self):
        return '111'
    value = property(get_bar)  #创建值为property对象的静态字段

obj = Foo()
result = obj.value       # 自动调用get_bar方法，并获取方法的返回值
print(result)
```

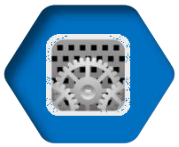
property的构造方法中有个四个参数

第一个参数是获取属性值的函数

第二个参数是是设置（修改）属性值的函数

第三个参数是是删除属性值的函数

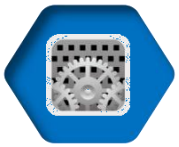
第四个参数是字符串，__doc__是该属性的描述信息



6.4 属性

```
class Student:
    def __init__(self):
        self.age1=None
    def get_age(self):
        return self.age1
    def set_age(self, value): # *必须两个参数
        self.age1=value
    def del_age(self):
        print( 'delete...')
        del self.age1
    age = property(get_age, set_age,del_age,'年龄为:')

stu = Student()
stu.age = 18    # 自动调用第二个参数的方法：set_age，将18当作参数传入
print(Student.age.__doc__,)    # 自动获取第四个参数中设置的值：
print(stu.age)  # 自动调用第一个参数中定义的方法：get_age
del stu.age    #自动调用第三个参数中定义的方法：del_age
```



6.4 属性

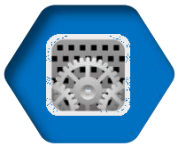
新式类（如果类继承自**object)**，class Goods(object):

装饰器方式针对经典类和新式类有所不同。

新式类具有三种**@property**装饰器

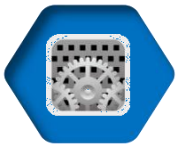
@property、**@方法名.setter**、**@方法名.deleter**

由于新式类中具有三种访问方式，我们可以根据他们几个属性的访问特点，分别将三个方法定义为对同一个属性：获取、修改、删除



6.4 属性

```
class Goods(object):
    def __init__(self):
        self.original_price = 100
        self.discount = 0.8
    @property
    def price(self):
        new_price = self.original_price * self.discount
        return new_price
    @price.setter
    def price(self, value):
        self.original_price = value
    @price.deleter
    def price(self):
        del self.original_price
obj = Goods()
obj.price        # 获取商品价格
obj.price = 200  # 修改商品原价
del obj.price    # 删除商品原价
```



6.4 属性

DMiC -HFUT-

当使用静态字段的方式创建属性时，经典类和新式类无区别

```
class Goods(object):
    def __init__(self):
        self.original_price = 100
       self.discount = 0.8
    def get_price(self):
        new_price = self.original_price * self.discount
        return new_price
    def set_price(self, value):
        self.original_price = value
    def del_price(self):
        del self.original_price
    PRICE = property(get_price, set_price, del_price, '价格属性描述...')

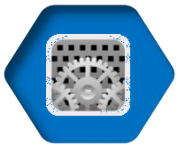
obj = Goods()
print(obj.PRICE)        # 获取商品价格
obj.PRICE = 200   # 修改商品原价
print(obj.PRICE)        # 获取商品价格
del obj.PRICE     # 删除商品原价
```

6.4 属性

- 在Python 3.x中，属性得到了较为完整的实现，支持更加全面的保护机制。
- 只读属性

```
>>> class Test:
    def __init__(self, value):
        self.__value = value
    @property
    def value(self):                    #只读，无法修改和删除
        return self.__value

>>> t = Test(3)
>>> t.value
3
>>> t.value = 5                         #只读属性不允许修改值
AttributeError: can't set attribute
>>> t.v=5                               #动态增加新成员
>>> t.v
5
>>> del t.v                             #动态删除成员
>>> del t.value                         #试图删除对象属性，失
败
AttributeError: can't delete attribute
>>> t.value
3
```



6.4 属性

- 可读、可写属性

```
>>> class Test:
    def __init__(self, value):
        self.__value = value

    def __get(self):
        return self.__value

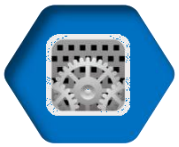
    def __set(self, v):
        self.__value = v

    value = property(__get, __set)

    def show(self):
        print(self.__value)
```

```
>>> t = Test(3)
>>> t.value        #允许读取属性值
3
>>> t.value = 5    #允许修改属性值
>>> t.value
5
>>> t.show()       #属性对应的私有变量也得到了相应的修改
5
>>> del t.value    #试图删除属性，失败

AttributeError: can't delete attribute
```



6.4 属性

- 可读、可修改、可删除的属性。

```
>>> class Test:
    def __init__(self, value):
        self.__value = value

    def __get(self):
        return self.__value

    def __set(self, v):
        self.__value = v

    def __del(self):
        del self.__value

    value = property(__get, __set, __del)

    def show(self):
        print(self.__value)
```

```
>>> t = Test(3)
>>> t.show()
3
>>> t.value
3
>>> t.value = 5
>>> t.show()
5
>>> t.value
5
```

```
>>> del t.value          #删除属性
>>> t.value              #对应的私有数据成员已删除
AttributeError: 'Test' object has no attribute
'_Test__value'
```



6.5 继承机制

➢ 继承(Inheritance)机制

- 继承是用来实现代码复用和设计复用的机制，是面向对象程序设计的重要特性之一。设计一个新类时，如果可以继承一个已有的设计良好的类然后进行二次开发，无疑会大幅度减少开发工作量。

- 在继承关系中，已有的、设计好的类称为父类或基类，新设计的类称为子类或派生类。派生类可以继承父类的公有成员，但是不能继承其私有成员。如果需要在派生类中调用基类的方法，可以使用内置函数super()或者通过“基类名.方法名()”的方式来实现这一目的。

- 构造函数、私有方法以及普通公有方法的继承原理。

```
class A(object):
    def __init__(self):  #构造方法可能会被派生类继承
        self.__private()
        self.public()
    def __private(self): #私有方法在派生类中不能直接访问
        print('__private() method in A')
    def public(self): #公有方法在派生类中可以直接访问，也可以被覆盖
        print('public() method in A')
```



6.5 继承机制

```
class B(A):    #类B没有构造方法，会继承基类的构造方法

    def __private(self):    #这不会覆盖基类的私有方法
        print('__private() method in B')

    def public(self):       #覆盖了继承自A类的公开方法public
        print('public() method in B')


>>> b = B()                          #自动调用基类构造方法
__private() method in A
public() method in B

>>> dir(b)                  #基类和派生类的私有方法访问方式不一样
['_A__private', '_B__private', '__class__', ...]
```

```
class C(A):
    def __init__(self):          #显式定义构造函数
        self.__private()         #这里调用的是类C的私有方法
        self.public()

    def __private(self):
        print('__private() method in C')

    def public(self):
        print('public() method in C')

>>> c = C()                      #调用类C的构造方法
__private() method in C
public() method in C

>>> dir(c)
['_A__private', '_C__private', '__class__', ...]
```

super()：返回超类的临时对象，允许调用其方法

```
class A:
    def __init__(self):
        self.x=1
        self.y=2
class B(A):
    def __init__(self):
        super().__init__()
        self.z=3

b1=B()
print(b1.x,b1.y,b1.z)
```

Python支持多继承(**Multiple Inheritance**)

- 多继承下的Method Resolution Order：
  - 如果父类中有相同的方法名，而在子类中使用时没有指定父类名，则Python解释器将从左向右按顺序进行搜索。

```
class A:
    def method(self):
        print('A')
class B:
    def method(self):
        print('B')
class C(A, B):
    pass
c = C()
c.method() # 输出什么？
```

A



6.6 多态原理与实现

- 所谓多态（polymorphism），是指基类的同一个方法在不同派生类对象中具有不同的表现和行为。派生类继承了基类行为和属性之后，还会增加某些特定的行为和属性，同时还可能会对继承来的某些行为进行一定的改变，这都是多态的表现形式。
- Python大多数运算符可以作用于多种不同类型的操作数，并且对于不同类型的操作数往往有不同的表现，这本身就是多态，是通过特殊方法与运算符重载实现的。



6.6 多态原理与实现

```
class Animal(object):       #定义基类
    def show(self):
        print('I am an animal.')
class Cat(Animal):          #派生类，覆盖了基类的show()方法
    def show(self):
        print('I am a cat.')
class Dog(Animal):          #派生类
    def show(self):
        print('I am a dog.')
class Tiger(Animal):        #派生类
     def show(self):
         print('I am a tiger.')
class Test(Animal):         #派生类，没有覆盖基类的show()方法
    pass

x = [item() for item in (Animal, Cat, Dog, Tiger, Test)]
for item in x:
    item.show()
```

```
I am an animal.
I am a cat.
I am a dog.
I am a tiger.
I am an animal.
```

6.7 本章小结

内容回顾

- 类的定义与使用
- 方法、属性
- **继承与多态**

思考

- 根据已学的几种数据结构编写Python类